SOL_AGENTS · MODULE GUIDE

# The Git Layer

How `tools/git.py` works, why every flag in it is there, and how the rest of the project fits around it. Written for someone who has not used `subprocess` before.

---

CONTENTS


1. What you are building
2. Map of the code
3. Background: running another program
4. Why the orchestrator needs Git
5. Walking through `tools/git.py`
6. Every guard, and what it prevents
7. How one call travels
8. The permission boundary
9. The modules next door
10. What is built, what is not
11. Glossary

## What you are building

Sol_Agents runs two language models against one software project. One model plays **Developer**: it proposes and writes code. The other plays **Reviewer**: it tries to prove the Developer's work is wrong. A third party — your Python code, called the **orchestrator** — sits above both, decides who runs when, and holds the authority neither model has.

The whole design rests on one rule: *the models are not trusted to report on their own work.* A model claiming "I fixed it and the tests pass" is a sentence, not evidence. So the orchestrator checks for itself — by reading files, running tests, and asking Git what actually changed on disk.

That last part is what `tools/git.py` is for.

## Map of the code

Every file in the project, and the one job it has.

| | |
|---|---|
| `config.py` | Reads settings from `.env` — which models to use, where the Ollama server is, and `PROJECT_PATH`: the folder the agents are allowed to touch. |
| `main.py` | The entry point. Asks you for a request, builds a specification, shows it to you for approval, then runs the Developer and Reviewer once each. |
| `schemas/requirements.py` | Defines what a valid specification looks like, using Pydantic. Not documentation — an enforced shape. |
| `orchestrator/specification.py` | Turns your plain-English request into numbered, testable requirements, and rejects any the model made up. |
| `agents/developer.py` | The Developer's system prompt and its connection to Ollama. |

| | |
|---|---|
| `agents/reviewer.py` | The Reviewer's, kept deliberately separate so the two never share instructions. |
| `tools/project.py` | Safe filesystem access: list, read, search — all locked inside `PROJECT_PATH`. |
| `tools/git.py` | Safe Git access: status, diff, log. **The subject of this document.** |
| `tools/toolsets.py` | Decides which of those tools each agent is allowed to have. |
| `tests/` | 52 tests. Mostly about proving the safety boundaries actually hold. |

## Background: running another program

Git is not a Python library. It is a separate program — `git.exe` on your machine. To use it from Python you have to start it as a *child process*, wait for it to finish, and collect what it printed.

Python's `subprocess` module does this. The core call looks like:

```
done = subprocess.run(["git", "status"], capture_output=True)
```

Three things come back from a finished program, and `git.py` keeps all three.

### `stdout`

"Standard output" — the program's normal answer. For `git diff`, this is the diff itself.

### `stderr`

"Standard error" — a *second, separate* stream for complaints and diagnostics. Keeping it separate means an error message can never get mixed into the middle of real output. When Git says `fatal: not a git repository`, that goes to stderr.

### `the return code`

A single number the program hands back when it exits. By universal convention **0 means success** and anything else means trouble. Git uses `128` for fatal errors. Note the trap: some commands use a non-zero code to mean

something perfectly normal — `git diff --quiet` returns `1` to say "yes, there are differences." That is why `git.py` never blindly treats non-zero as failure.

### The list, and why it matters more than it looks

Notice the command was written as a *list*: `["git", "status"]`. You can instead pass one string with `shell=True`, which hands the text to the system shell first.

Never do that here. A shell interprets punctuation. `;` ends a command and starts another. `|` pipes. `$(...)` substitutes the output of yet another command. So if any part of that string came from somewhere you do not control — and in Milestone 3, arguments start coming *from the language model* — a filename like `a.py; del *` stops being a filename and becomes a second command.

With a list and no shell, each item is handed to the operating system as one complete argument. Punctuation inside it is just characters. This is the single most important habit in the whole module.

> **WORTH INTERNALISING**
>
> The list form is not a style preference. It is the boundary between "Git reads a strangely-named file" and "an attacker runs a command on your machine." Build the habit before untrusted input arrives, not after.

---

## Why the orchestrator needs Git

Imagine the Developer has just finished an iteration. You need to answer one question before the Reviewer can do anything useful: **what actually changed?**

You could ask the Developer. It would tell you — plausibly, fluently, and sometimes wrongly. Models routinely describe edits they did not make.

Git already knows. It tracks every file in the project and can report precisely which lines differ from the last commit. It does not have opinions and it cannot be persuaded. The workflow document puts it plainly: *"Git should act as the code-history memory of the system. Model conversation history is not a reliable audit mechanism."*

So `git.py` gives the orchestrator three abilities:

**See the current state** — `status()` lists which files are modified but not yet committed.
**See exactly what changed** — `diff()` produces the line-by-line difference, which becomes the Reviewer's primary input.
**Mark a point in time** — `current_commit()` records where a run started, so you can later ask "what changed since then" and, if an iteration goes badly, roll back to it.

# Walking through `tools/git.py`

## The evidence record

```
@dataclass(frozen=True)
class GitResult:
    command: list[str]
    returncode: int
    stdout: str
    stderr: str
    stdout_truncated: bool = False
    stderr_truncated: bool = False

    @property
    def ok(self) -> bool:
        return self.returncode == 0

    @property
    def truncated(self) -> bool:
        return self.stdout_truncated or self.stderr_truncated
```

Every Git call returns one of these.

`@dataclass` is a Python shortcut. You list the fields you want; Python writes the boilerplate — the constructor, a readable printout, and equality comparison — for you. Without it you would hand-write about twenty lines that do nothing interesting.

`frozen=True` makes each instance *immutable*: once created, assigning to a field raises an error instead of quietly changing it. That is a deliberate choice, not caution for its own sake. This object is a *record of what happened*. If later code could edit the exit code or rewrite the output, it would stop being evidence and become a rumour.

`@property` turns a method into something you read like a plain attribute — you write `result.ok`, not `result.ok()`. It is computed fresh each time from the fields, so it can never drift out of step with them.

### Why `command` is stored at all

This is the field beginners delete, and it is the most important one. The workflow document requires that the orchestrator capture *"the actual command, exit code, stdout/stderr."* A log saying "the diff was empty" is worth very little. A log saying `git --no-pager -C D:\Demos diff --no-color --no-ext-diff HEAD` → exit 0 → empty is something you can re-run yourself six months later and check.

Returning a bare string from these functions would throw away the exit code, the error stream, and the command — three quarters of the evidence.

## Cutting output down to size

```
def _truncate(text: str) -> tuple[str, bool]:
    encoded = text.encode("utf-8")

    if len(encoded) <= MAX_OUTPUT_BYTES:
        return text, False

    clipped = encoded[:MAX_OUTPUT_BYTES].decode(
        "utf-8",
        errors="ignore",
    )

    omitted = len(encoded) - MAX_OUTPUT_BYTES

    return (
        f"{clipped}\n\n"
        f"... [output truncated, {omitted} bytes omitted]",
        True,
    )
```

Returns the possibly-shortened text, plus whether it was shortened.

A diff can be enormous. Everything here eventually goes into a prompt, and a model can only hold so much text at once, so there has to be a ceiling.

**Why bytes and not characters.** A Python string is a sequence of *characters*. What travels through a pipe is *bytes*. In UTF-8 encoding, an ordinary English letter takes one byte, but an accented letter or an emoji takes two to four. So "200,000 characters" and "200,000 bytes" are different limits. `.encode("utf-8")` converts the string to bytes so the budget is measured in the unit that actually matters.

**Why `errors="ignore"`.** Slicing bytes at position 200,000 can land in the *middle* of a multi-byte character, leaving a fragment that is not valid UTF-8. `errors="ignore"` tells Python to drop that broken fragment instead of raising. It is a deliberate, tiny loss at exactly one place.

**Why it returns two things.** `tuple[str, bool]` means the function hands back a pair: the text, and a flag saying whether anything was cut. The caller unpacks it in one line:

```
stdout, stdout_truncated = _truncate(done.stdout or "")
```

> **WHY THE FLAG EXISTS**
>
> Silent truncation is not a performance detail — it is a correctness bug. A Reviewer handed a quietly shortened diff reviews half a change and reports that it looks fine. The visible marker tells a human; the boolean lets the orchestrator refuse to proceed on a partial diff.
>
> This is also why stdout and stderr get separate flags. A noisy-but-clipped error stream should not make an intact diff look untrustworthy.

## The engine

```
def _run_git(*args: str) -> GitResult:
    command = [
        "git",
        "--no-pager",
        "-C",
        str(get_project_path()),
        *args,
    ]

    try:
        done = subprocess.run(
            command,
            capture_output=True,
            timeout=GIT_TIMEOUT,
            encoding="utf-8",
            errors="replace",
        )

    except FileNotFoundError as exc:
        raise GitError(
            "Git executable not found on PATH."
        ) from exc

    except subprocess.TimeoutExpired as exc:
        raise GitError(
            f"Git timed out after {GIT_TIMEOUT}s: "
            f"{' '.join(command)}"
        ) from exc
```

Every public function in the module goes through here.

The leading underscore in `_run_git` is a Python convention meaning "internal — not part of what this module offers to the outside world."

`*args: str` means the function accepts any number of string arguments. Calling `_run_git("status", "--branch")` makes `args` the tuple `("status", "--branch")`. The `*args` inside the list literal does the reverse: it spreads those items into the list one by one.

### The most important line in the file

`"-C", str(get_project_path())` tells Git which repository to operate on.

Without it, Git uses whatever folder the Python process happens to be sitting in — which is `Sol_Agents`, the orchestrator's own repository. Here is that exact mistake, measured:

```
git with no -C   -> D:/MCP_Sol/Sol_Agents   ← the orchestrator itself
git -C PROJECT   -> D:/Demos                ← the project under review
```

Both are real Git repositories, so nothing errors. The Reviewer would receive a perfectly valid diff of the wrong codebase and review your orchestrator instead of the project. That is the kind of bug that survives for weeks.

You could instead pass `cwd=root` to `subprocess.run`, which also works. `-C` is better for one specific reason: it puts the target repository *inside the recorded command*. With `cwd=`, the single most important fact about the call lives outside the evidence you saved.

### The keyword arguments, one at a time

| | |
|---|---|
| `capture_output=True` | Collect stdout and stderr instead of letting them print to your terminal.<br>**WITHOUT IT:** The output vanishes into the console and your code receives nothing. |
| `timeout=GIT_TIMEOUT` | Give up after 30 seconds and raise.<br>**WITHOUT IT:** Git can block indefinitely waiting on `index.lock`. Your program hangs with no way out but Ctrl-C. |

| | |
|---|---|
| `encoding="utf-8"` | Decode the bytes as UTF-8.<br><br>**WITHOUT IT:** Python falls back to your Windows locale (cp1252). Any non-English character in a source file comes back mangled, and the Reviewer reports the corruption as a bug in the code. |
| `errors="replace"` | Substitute a placeholder for anything that will not decode.<br><br>**WITHOUT IT:** One stray byte anywhere in a large diff raises an exception and you lose the entire result. |

#### Turning crashes into one kind of error

Two things can go wrong before Git even runs, and both are caught and re-raised as `GitError`:

`FileNotFoundError` means `git` is not installed or not on your PATH. Python's default message for this is cryptic; the replacement says exactly what is wrong.

`TimeoutExpired` means Git started but never finished. The replacement message includes the full command, so you know what was stuck.

The `from exc` at the end matters. It *chains* the exceptions, so the traceback shows both your clear message and the original technical cause underneath. You get the friendly explanation without destroying the evidence.

Giving both failures a single type, `GitError`, means calling code can write one `except GitError:` and handle every way Git can fail to run — while a genuine bug in your own code still crashes loudly instead of being swallowed.

## The six public functions

#### is_git_repository() and require_git_repository()

```
result = _run_git("rev-parse", "--is-inside-work-tree")
return result.ok and result.stdout.strip() == "true"
```

`rev-parse --is-inside-work-tree` is Git's own question-asking command. It prints `true` if the folder is inside a repository. The check tests *both* that the command succeeded *and* that it printed the right word — either alone would be a weaker test.

`require_git_repository()` wraps it and raises instead of returning `False`. Every public function calls it first, so pointing `PROJECT_PATH` at an ordinary folder gives you one clear sentence naming the path, rather than a stream

of confusing Git errors from every command you try.

### `status()`

```
_run_git("status", "--porcelain=v1", "--branch")
```

Real output from a repository with one edited file:

```
## master
 M a.py
```

`--porcelain` is Git's name for machine-readable output — stable, terse, and *unaffected by the user's configuration*. The human-readable version of `git status` changes wording between Git versions and can be translated into other languages; parsing it is a trap. The `=v1` pins the format version, so a future Git upgrade cannot silently change what your parser sees. `--branch` adds the `##` line naming the current branch.

### `diff()`

```
def diff(base: str | None = "HEAD") -> GitResult:
    require_git_repository()

    args = ["diff", "--no-color", "--no-ext-diff"]

    if base is not None:
        if base.startswith("-"):
            raise GitError(f"Invalid base ref: {base!r}")
        args.append(base)

    return _run_git(*args)
```

Output for a one-line change:

```
-x = 1
+x = 2
```

Three decisions are packed into those few lines.

**The default is `"HEAD"`, not nothing.** Plain `git diff` shows only changes that have *not been staged*. The moment the orchestrator stages a file or commits a checkpoint, plain `git diff` returns empty — and the Reviewer, seeing

nothing, concludes the work is clean. Comparing against `HEAD` catches staged and unstaged changes together. Passing a commit ID instead lets you ask "what changed since this run began," which is what checkpointing needs.

**Refs starting with `-` are rejected.** This is subtle and worth slowing down for. Using a list instead of a shell stops an argument becoming a *command*. It does not stop an argument becoming a *flag*. Git accepts options like `--upload-pack=...` that name a program to execute. If a base ref ever arrives from the model — and in Milestone 4 it will — a value starting with `-` would be read as an option rather than a branch name. One `startswith` closes it.

**Two flags neutralise the user's own Git config.** Those are covered below with the rest.

### `recent_commits()` and `current_commit()`

```
_run_git("log", f"-{count}", "--oneline", "--no-color", "--no-decorate")
```

`--oneline` gives one short line per commit — `6dc6cf4 Initial demo project` — which is what you want in a prompt rather than full commit bodies. `count` is checked to be between 1 and 100 *before* it reaches the command line, so an absurd value fails fast with a clear message instead of being silently clipped by the truncator later.

`current_commit()` is the one function returning a plain string rather than a `GitResult`, because a commit ID is either read successfully or is an error — there is no partial answer. It raises if Git failed, so callers never receive an empty string and treat it as a real ID.

---

## Every guard, and what it prevents

Gathered in one place: each defensive choice in the module, and the specific thing that goes wrong without it.

| | |
|---|---|
| `list of arguments, never shell=True` | Each item reaches the OS as one argument; punctuation stays literal.<br>**WITHOUT IT:** Text from the model can inject a second command. |
| `-C <project path>` | Names the target repository explicitly, and records it in the evidence.<br>**WITHOUT IT:** Git operates on the orchestrator's own repo. No error — just a valid diff of the wrong codebase. |

| | |
|---|---|
| `--no-pager` | Never invoke a pager program.<br>**WITHOUT IT:** An unusual pager configuration could intercept output. Belt-and-braces, since piped output already disables paging. |
| `--no-color` | Plain text, whatever the user's config says.<br>**WITHOUT IT:** A `color.ui = always` setting embeds ANSI escape codes in the model's prompt, which it may then report as bugs. |
| `--no-ext-diff` | Use Git's own diff, not an external program.<br>**WITHOUT IT:** A `diff.external` setting makes Git run an arbitrary program — unpredictable output, and config-driven code execution. |
| `--porcelain=v1` | Stable, machine-readable, version-pinned status output.<br>**WITHOUT IT:** Wording varies by Git version and language; a parser built on it breaks silently after an upgrade. |
| `--no-decorate` | Omit branch and tag annotations from the log.<br>**WITHOUT IT:** Extra text varies with local branches, making output that should be stable inconsistent. |
| `base.startswith("-")` | Rejects a ref that would be parsed as a command-line option.<br>**WITHOUT IT:** Argument injection — a model-supplied ref becomes a Git flag. |
| `1 <= count <= 100` | Bounds the log size at the boundary.<br>**WITHOUT IT:** An enormous request is silently clipped later instead of failing clearly now. |
| `require_git_repository()` | One clear failure naming the path.<br>**WITHOUT IT:** Every command emits its own confusing Git error and you debug the wrong thing. |

| | |
|---|---|
| `visible truncation marker` | Says in the text itself that output was cut.<br>**WITHOUT IT:** The Reviewer approves a change it only partly saw. |
| `frozen=True` | The evidence record cannot be edited after the fact.<br>**WITHOUT IT:** An audit trail that downstream code can quietly rewrite is not an audit trail. |

Notice how many of these defend against **the user's own Git configuration** rather than against an attacker. Git is unusually configurable, and a setting in a `.gitconfig` file you have forgotten about can change what your program receives. Pinning the output format is how you make the module behave the same on every machine.

## How one call travels

This is a genuine sequence, so it is worth following end to end. When the orchestrator asks for a diff:

```
1  toolsets.py — ReadOnlyToolset.git_diff()
```

The agent's toolset receives the call. This is the permission gate: the method exists here, so this agent is allowed to ask.

2 git.py — diff()

Checks it is a real repository, rejects a base ref starting with `-`, assembles the argument list.

3 git.py — _run_git()

Prepends `git --no-pager -C <path>`, taking the path from `get_project_path()` in `project.py`, which reads `PROJECT_PATH` from your `.env`.

4 `subprocess.run()`

Python starts `git.exe` as a separate process with those exact arguments and waits, with a 30-second limit.

5 git.exe

Git reads the repository on disk and writes the diff to stdout, anything unusual to stderr, then exits with a return code.

6 git.py — _truncate()

Both streams are measured in bytes and shortened if over budget, each with its own flag.

7 git.py — GitResult

Command, return code, both streams and both flags are frozen into one record and handed back.

Seven steps, and only step five touches the disk. Everything either side of it exists to make that one step safe, bounded, and recorded.

---

## The permission boundary

The workflow document is strict on one point: the Reviewer must never modify the code it is reviewing. A Reviewer that can edit the work destroys the independence the entire two-agent design depends on.

Saying so in a prompt is not enough. Prompts are requests, and models do not always comply. So the restriction is built into the shape of the code.

```
class ReadOnlyToolset:
    """Project inspection only. Nothing here can modify the project."""

    def list_files(self) -> list[str]: ...
    def read_file(self, relative_path) -> str: ...
    def search_files(self, query) -> list[dict]: ...
    def git_status(self) -> GitResult: ...
    def git_diff(self, base="HEAD") -> GitResult: ...
    def git_recent_commits(self, count=10) -> GitResult: ...


class DeveloperToolset(ReadOnlyToolset):
    """Everything the Reviewer can do. Write access is added here in M3."""
```

Bodies elided. The point is which class each method sits on.

The orchestrator builds one of each and hands each agent only its own:

```
developer = DeveloperAgent(DeveloperToolset())
reviewer  = ReviewerAgent(ReadOnlyToolset())
```

When `write_file` is added in Milestone 3, it goes on `DeveloperToolset` only. If the Reviewer then asks to write something, the orchestrator looks at the object it holds, finds no such method, and refuses. The Reviewer is not *trusted* not to write — it is *unable* to.

Git sits on the read-only base because both roles need it and nothing here can change the repository. There is no `commit`, no `checkout`, no `push` — only questions.

> **THE GENERAL PRINCIPLE**
>
> An `import` is not a permission. If both agents import the same module, both can call everything in it. Handing each agent a restricted object is what turns a rule you wrote down into a rule the code enforces.

---

# The modules next door

## tools/project.py — the path guard

Everything Git does is bounded by `PROJECT_PATH`. Everything the *filesystem* tools do is bounded by one function:

```
requested_path = (root_path / relative_path).resolve()

if root_path not in requested_path.parents:
    raise ProjectAccessError(...)
```

`.resolve()` turns a messy path into its true, absolute form — following `..` steps and any shortcuts. Only then is the result checked to be underneath the project root. Doing it in that order is what defeats `../../etc/passwd`: the trickery is flattened out *before* the check, not after.

It also defeats a folder shortcut pointing somewhere outside the project, because `.resolve()` follows those too and reports where they really lead. There are tests for exactly that case.

## orchestrator/specification.py — refusing invented requirements

A language model asked to list requirements will add helpful ones you never asked for. The specification builder makes that structurally difficult: every requirement must quote the exact words from your request that justify it, and the quote is checked against your *real* prompt.

Three separate rules apply to that quote — it must be at least two words, it must actually appear in your request, and it must land on whole-word boundaries rather than in the middle of longer words. If any fails, the specification is rejected and the model is asked again with the specific reason.

# What is built, what is not

The workflow document lays out milestones M0 through M10. Here is the honest position.

| MILESTONE | STATE | NOTES |
|---|---|---|
| M0 — Safe project access | NEARLY | Path guard, read tools and Git layer all done and tested. Git module has no tests of its own yet. |
| M1 — Two model calls | DONE | Separate prompts, separately configurable models. |
| M2 — Structured schemas | PARTIAL | Requirements exist. Review, findings and decisions do not. |
| M3 — Developer tool loop | NOT STARTED | Needs writes, plus a loop letting the model request tools. |
| M4 — Reviewer tool loop | NOT STARTED | — |
| M5 — Human checkpoints | PARTIAL | Specification approval only. |
| M6 — Adjudication | NOT STARTED | The Developer accepting or rejecting each finding. |
| M7 — Checkpoints and audit | NOT STARTED | `git.py` is the foundation this needs. |

## One thing to understand about today

Every read tool described here works and is tested. Almost none of them are *called*.

Both agents currently use exactly one method — `list_files()` — and paste the resulting filenames into their prompt. The model sees that `calculator.py` exists. It never sees a line inside it, and it cannot ask.

That is not an oversight. For a model to request a file, it must be able to say so, and that requires passing tool definitions to the chat call and looping on the response: the model asks, the orchestrator runs the tool through that agent's toolset, the result goes back, and round again. That loop is Milestone 3.

What exists now is the layer underneath it — safe, bounded, recorded access, with the permission split already in place, waiting for a caller.

# Glossary

**`argument injection`**

Supplying a value that gets read as a command-line option rather than data. Distinct from command injection, and not prevented by avoiding the shell.

**`child process`**

A separate program your program starts and waits for.

**`dataclass`**

A Python decorator that generates a class's constructor and comparison methods from a list of declared fields.

**`exit / return code`**

A number a finished program hands back. Zero means success by convention.

**`HEAD`**

Git's name for the commit you are currently on.

**`immutable`**

Cannot be changed after creation.

**`orchestrator`**

The plain Python code that owns the workflow. Neither model controls it.

**`porcelain`**

Git's term for machine-readable output, as opposed to output formatted for humans.

**`ref`**

Any name for a commit — a branch, a tag, `HEAD`, or a raw ID.

**`staged`**

Marked for inclusion in the next commit, via `git add`, but not yet committed.

**`stdout / stderr`**

The two separate output streams every program has: normal results, and diagnostics.

### UTF-8

The standard way of representing text as bytes. Common characters take one byte; others take up to four.

---

Sol_Agents module guide · covers `tools/git.py`, `tools/toolsets.py` and `tools/project.py` as of 18 September 2026 · 52 tests passing